

CD339A—F9: Metoder

Malmö universitet

Institutionen för datavetenskap och medieteknik

2025-09-22

Innehåll

Repetition

Om metoder

Vad är en metod?

Varför använda metoder?

Hur man skapar metoder

Skapa och anropa metoder

Parametrar och argument

Metodhuvud

Metodkropp

Return

Mer om metoder

Överlagring av metoder

Scope och call stack

Sammanfattning

Sammanfattning

Rekommenderad läsning

Repetition

Repetition

Inbyggda metoder

Vi har tidigare använt inbyggda metoder i Java. Exempelvis:

- ▶ `System.out.println()`
- ▶ `String.length()`
- ▶ `Math.sqrt()`
- ▶ `Integer.parseInt()`

Exempel

En liten kodsnuft som använder Pythagoras sats

```
1 public static void main(String[] args){  
2     //  $a^2+b^2=c^2$  (Pythagoras sats)  
3     double a = 3;  
4     double b = 4;  
5     double c = Math.sqrt(Math.pow(a,2)+Math.pow(b,2));  
6     System.out.println("Om kateterna är " + a + " och " + b + "  
7         är hypotenusan " + c);  
}
```

Här använder vi tre olika metoder: `Math.sqrt()`, `Math.pow()`, och `System.out.println()`

Om metoder

Vad är en metod?

Varför använda metoder?

Metod vs funktion

Några av er har säkert stött på begreppet *funktion* i tidigare programmeringsspråk. När vi pratar om *metoder* i Java så är det i praktiken samma sak som funktioner.

Nu i början kommer vi att hantera våra metoder som om de vore funktioner.

Vad är en metod?

En metod är en samling kommandon som man kan exekvera med ett anrop.

```
1 public static double max(double a, double b){  
2     if (a >= b){  
3         double greatest = a;  
4     }  
5     else{  
6         double greatest = b;  
7     }  
8     return greatest;  
9 }
```

Nu hade vi kunnat anropa funktionen med `double c = max(5.2, 7.9)`

Varför använda metoder?

Det finns flera anledningar till att vi vill arbeta med metoder

- ▶ Det ökar läsbarheten (detta kan inte överskattas)
- ▶ Koden blir lättare att felsöka
- ▶ Vi behöver inte upprepa kod i onödan
- ▶ Lättare att återanvända kod i andra projekt

Hur man skapar metoder

Skapa och anropa metoder
Parametrar och argument
Metodhuvud
Metodkropp
Return

Skapa egna metoder

När du skapar en metod i Java (och flera andra språk) så är det ett par saker vi behöver ange.

- ▶ Tillgänglighet
- ▶ Returvärde
- ▶ Metodnamn
- ▶ Parametrar

Vi kommer att ta upp *returvärde*, *metodnamn*, och *parametrar* idag. *Tillgänglighet* tar vi upp när vi pratar objekt.

Skapa egna metoder

```
1 public static char myMethod(String s){  
2     char firstLetter = s.charAt(0);  
3     return firstLetter;  
4 }
```

- ▶ `public` anger tillgänglighet
- ▶ `static` styr vad den får göra
- ▶ `char` anger vad metoden *returnerar*
- ▶ `myMethod` metodens namn
- ▶ `(String s)` parametrar som metoden tar emot
- ▶ `return firstLetter` skickar tillbaka `firstLetter`

Anropa metoder

```
1 public static void main(String[] args){
2     String x = "All that is gold does not glitter";
3     char a = myMethod(x); // Anropa metoden, spara svaret i a
4     System.out.println(a);
5 }
6 public static char myMethod(String s){
7     char firstLetter = s.charAt(0); // Hitta första bokstaven
8     return firstLetter; // Skicka tillbaka första bokstaven
9 }
```

Parametrar

En metods *parametrar* är det som metoden tar emot i metodhuvudet. I exemplet på den tidigare bilden var det en sträng med namnet *s*. Metoder kan ta emot flera parametrar

```
1 public static double max(double a, double b){  
2     double greatest;  
3     if (a > b){  
4         greatest = a;  
5     }  
6     else{  
7         greatest = b;  
8     }  
9     return greatest;  
10 }
```

Parametrar

Parametrarna kan vara olika datatyper

```
1 public static String secondMethod(String a, char b){  
2     String newString = a+b; // Konkatererar texten  
3     return newString; // Returnerar vår nya sträng  
4 }
```

Argument

Ett *argument* är värdet vi skickar in till en metod när vi anropar metoden.

```
1 public static void main(String[] args){  
2     String s = secondMethod("Wow", 'a');  
3     System.out.println(s);  
4 }
```

Här är "Wow" och 'a' argument som vi skickar in till metoden.

argument → parameter

Returvärde

Java är ett strikt typat språk, vilket betyder att man behöver hålla koll på variabelers datatyper. Det gör att vi vill att en metod alltid returnerar samma datatyp. Detta anger vi i metodhuvudet.

```
1 public static String secondMethod(String a, char b){  
2     String newString = a+b; // Konkaterar texten  
3     return newString; // Returnerar vår nya sträng  
4 }
```

Här ser vi att `secondMethod` anger att den skickar tillbaka en sträng.

I vårt tidigare exempel `max` returnerade vi alltid en `double`.

Void-metoder

Det finns tillfällen när en funktion inte ska returnera något. Då anger man detta med `void`

```
1 public static void greet(String name, int age){  
2     System.out.println("Hello "+name+", it's so cool that you  
        are "+age+" years old!")  
3 }
```

Statiska metoder

Statiska metoder är lite speciella på så vis att de är kopplade direkt till klassen och inte till en instans av en klass.

Vi kommer att använda statiska metoder fram tills att vi börjar jobba med klasser.

Läs mer i Deitel på sida 207.

Metodkropp

Med *metodkropp* menar vi det som kommer mellan måsvingarna.

Det är i metodkroppen vi skriver all kod som ska exekveras.

```
1 public static String secondMethod(String a, char b){  
2     String newString = a+b; // Konkaterar texten  
3     return newString; // Returnerar vår nya sträng  
4 }
```

I `secondMethod` är det raderna 2 och 3 som utgör metodkroppen.

Metodkropp

Metodkroppen kan vara hur lång som helst. Men en rekommendation är att hålla den kort. Skulle den bli lång kan man fundera på om man kanske ska bryta ut delar av metodkroppen till egna metoder.

En tumregel är att man ska kunna se hela metoden utan att behöva scrolla på skärmen. *Detta går inte alltid att följa.*

Kodningsfilosofi

Försök att skapa metoder som gör en sak.

Ge metoderna korta men beskrivande namn. Om du inte kan ge den ett kort namn fundera på om det borde vara flera metoder istället.

Metoder kan anropa andra metoder

Metod som anropar andra metoder

Här har vi ett exempel på en metod `checkSensors` som anropar flera andra metoder.

```
1 public void checkSensors() {  
2     checkDoorSensors();  
3     checkStaircaseSensors();  
4     checkLobbySensors();  
5     checkOfficeSensors();  
6 }
```

Genom att dela upp `checkSensors()` i flera mindre metoder så är det fortfarande tydligt vad metoden gör.

Return

Kommandot `return` skickar tillbaka det värde som kommer direkt efter på samma rad.

```
1 public static String secondMethod(String a, char b){  
2     String newString = a+b; // Konkatenerar texten  
3     return newString; // Returnerar vår nya sträng  
4 }
```

Här är det `newString` som skickas tillbaka till anropet.

Flera return

Tidigare hade vi följande exempel:

```
1 public static double max(double a, double b){  
2     double greatest;  
3     if (a > b){  
4         greatest = a;  
5     }  
6     else{  
7         greatest = b;  
8     }  
9     return greatest  
10 }
```

Flera return

Vi kan korta ner den koden, och göra den läsligare:

```
1 public static double max(double a, double b){  
2     if (a > b){  
3         return a; // Returnerar a  
4     }  
5     else{  
6         return b; // Returnerar b  
7     }  
8 }
```

Flera return

När programmet når en return så avslutas metoden och värdet efter returneras.

Koden kommer alltså inte att fortsätta och returnera något mer senare.

Mer om metoder

Överlagring av metoder
Scope och call stack

Överlagring av metoder

Det händer att man har en metod där man vill att den ska kunna ta emot olika parametrar. Kanske för att man vill kunna ta emot olika mängder av parametrar eller för att man vill göra lite olika saker beroende på vad som skickas in.

Det är möjligt att skapa flera metoder med samma namn, förutsatt att de alla returnerar samma datatyp, och att de har olika parameterlistor.

Överlagring av metoder

Exempel

```
1 public double max(double a, double b){  
2     if (a>=b){return a;} // Vertical space is prime real estate  
3     else{return b;}  
4 }  
5 public double max(double a, double b, double c){  
6     if (a>=b && a>=c){return a;}  
7     else if (b>=a && b>=c){return b;}  
8     else{return c;}  
9 }
```

Nu gör kommandona `max(1.0,3.0)` och `max(5.0,3.0,7.0)` lite olika saker

Överlagring av metoder

Exempel

```
1 public double max(double a, double b){  
2     if (a>=b){return a;} // Vertical space is prime real estate  
3     else{return b;}  
4 }  
5 public double max(double a, double b, double c){  
6     return max(a, max(b, c));  
7 }
```

Hur fungerar det här?

Överlagring av metoder

Exempel

```
1 public double max(double a, double b){  
2     if (a>=b){return a;} // Vertical space is prime real estate  
3     else{return b;}  
4 }
```

Vad händer om man skickar in två `int` till `max()` nu?

Överlagring av metoder

Exempel

Skulle vi ändra på parametrarna till `int` istället och skicka in två `double` istället, vad händer då?

```
1 public double max(int a, int b){  
2     if (a>=b){return a;} // Vertical space is prime real estate  
3     else{return b;}  
4 }
```

Scope

Som regel kan en metod inte komma åt variabler som skapats utanför metoden (detta är särskilt sant när man använder `static`)

På samma sätt kan andra metoder inte komma åt variabler som instansierats i en annan metod.

Detta skapar en sorts inkapsling och decentralisering som gör att varje metod bara gör sin grej.

Man säger ibland att variablerna är *lokala*.

Call stack

När du anropar en funktion så skapas något som kallas för en *stack frame*.

En *stack frame* håller koll på initialiserade variabler och vart de pekar i minnet.

När en metod avslutas så returnas eventuellt return värde och sen förstörs *stack framen* och alla lokala variabler glöms bort ur minnet.

Call stack

Lite djupare

För att förstå hur *call stacken* fungerar behöver man förstå hur *stackar* fungerar.

En stack är en datastruktur som lagrar data enligt modellen *Last-in-First-out* (*LiFo*).

Vi kommer inte att diskutera stackar på djupet i den här kursen.

Call stack

Lite djupare

Förenklat kan vi jämföra en stack med en hög med disk. Du lägger på ny disk överst i högen, och diskar den översta disken först.

Likt diskhögen hamnar den senast anropade metoden överst. När den avslutas så kastas det som är kvar i *ramen* och programmet fortsätter med den näst översta *ramen* i stacken.

När den sista *ramen* är avslutad så är vår `main`-metod avslutad och programmet stängs ner.

Sammanfattning

Sammanfattning
Rekommenderad läsning

Sammanfattning

- ▶ metoder ökar läsligheten av vår kod
- ▶ metoder ska vara specifika och namnade på ett beskrivande sätt
- ▶ metoder returnerar alltid samma typ av värden
- ▶ en metod kan vara void och inte returnera något alls
- ▶ i metodhuvudet anger vi tillgängligheten, datatyp som returneras, metodnamn och parametrar
- ▶ vi anropar metoder genom att skriva metodnamnet och skicka med argument
- ▶ en metod avslutas när den når **return**

Sammanfattning

- ▶ **metoder ökar läsligheten av vår kod** (om någon skulle glömt det)
- ▶ metoder kan *överskugga* varandra om de har samma namn men olika parameter-listor
- ▶ när vi anropar en metod skapas en ny *stack frame*
- ▶ en *stack frame* håller reda på alla lokala variabler

Rekommenderad läsning

Deitel & Deitel, kapitel 5, sidorna 204–236

- ▶ <https://docs.oracle.com/javase/tutorial/java/java00/methods.html>
- ▶ <https://docs.oracle.com/javase/tutorial/java/java00/arguments.html>
- ▶ <https://docs.oracle.com/javase/tutorial/java/java00/returnvalue.html>

Cornelia Funke, *The Colour of Revenge*